

## 1.What is CRUD in Fast API?

CRUD stands for Create, Read, Update, and Delete. These are the basic operations performed in database and in Fast API, CRUD operations are implemented by using HTTP methods like POST, GET, PUT, and DELETE. For example, adding a student uses POST, and deleting a student uses DELETE. It may helps to manage data efficiently in APIs.

## 2. Why do we use the POST method for inserting data?

The POST method is used to send new data to the server and also used for creating new records in databases or applications. In Fast API, POST receives data through the request body. It is preferred because it securely sends data to the server. For example, adding a new student record uses POST /students.

## 3. Difference between PUT and PATCH

PUT is used to completely update an existing record. In this all fields are usually sent again in the request body. It replaces data completely, while PATCH modifies only selected values. PATCH is used to partially update only specific fields of a record and only the fields that need changes are sent.

## 4. What is the path parameter?

A path parameter is a value passed inside the URL path and also it helps identify a specific resource or record.

In FastAPI, path parameters are defined inside curly braces {}. It is commonly used for fetching, updating, or deleting records by ID.

Example: /students/{student\_id} where student\_id is the path parameter.

## 5. What is the request body in FastAPI?

In Fast API, request body data is usually sent in JSON format. It is commonly used in POST and PUT methods. It automatically converts JSON data into Python objects using Pydantic models.

Example: student details like name, age, and marks are sent in the request body.

## 6. Why do we use Pydantic models?

Pydantic models are used for data validation in Fast API.

They ensure that the incoming data matches the required format and datatype.

If invalid data is sent, FastAPI automatically shows an error message.

Pydantic also helps in serialization and documentation generation.

It improves code readability and reduces manual validation work.

## 7. What status code is returned for successful deletion?

The commonly used status code for successful deletion is `200 OK`.

Sometimes APIs also return `204 No Content` after deletion.

In FastAPI, developers can customize the response status code.

A successful delete operation confirms that the resource was removed properly.

It helps clients understand that the request was completed successfully.

## 8. How can we check whether a student exists before updating?

Before updating, we search the student list using the student ID.

If the student ID is found, the record is updated.

If the ID does not exist, Fast API raises an `HTTPException`.

Usually, status code `404 Not Found` is returned.

This prevents updating invalid or non-existing records.

## 9. What happens if duplicate IDs are inserted?

Duplicate IDs can create confusion and data inconsistency.

Multiple students may have the same identifier, making retrieval difficult.

To avoid this, we should check existing IDs before insertion.

If a duplicate ID is found, an error message should be returned.

Unique IDs help maintain accurate and organized data.

## 10. How does Fast API automatically generate Swagger UI?

Fast API automatically generates Swagger UI using Open API standards.

It reads all API routes, request models, and response models.

The documentation is created dynamically without extra coding.

Swagger UI helps developers test APIs directly from the browser.

It is available by default at `/docs` in Fast API applications.